PRODUCT MANAGEMENT SYSTEM

Django Practical Exam Guide

Goal: Create a Django project, build a Product model, register it in Django Admin, and add, edit, and delete products.

Before starting: Python and Django should be installed. Open PowerShell or the VS Code terminal.

Remember: Run commands in the terminal from the project folder. File contents must be typed into the named Python files, not into the terminal.

Part A — Create the project

1. Create a folder on the Desktop using your roll number. Open that folder in VS Code, then open Terminal → New Terminal (or PowerShell).

2. Create the Django project:

```
python -m django startproject ProductManagementSystem .
```

The dot (**.**) at the end is important. It creates the project in the current folder.

3. Create the app:

```
python manage.py startapp products
```

Part B — Create the Product model

4. Open **products/models.py**, remove its existing contents if needed, and enter:

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    category = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    quantity = models.IntegerField()

    def __str__(self):
        return self.name
```

Part C — Register the app and model

5. Open **ProductManagementSystem/settings.py**. Find **INSTALLED_APPS** and add **'products',** inside the list, for example:

```
INSTALLED_APPS = [
    # existing Django apps...
    'products',
]
```

Keep all the existing Django apps in the list. Add 'products' as one more item; do not replace the whole list.

6. Open **products/admin.py** and enter:

```
from django.contrib import admin
from .models import Product

admin.site.register(Product)
```

Part D — Create the database tables

7. Create migration files:

```
python manage.py makemigrations
```

8. Apply migrations to the database:

```
python manage.py migrate
```

If the commands finish without errors, Django has created the required database tables.

Part E — Create admin login

9. Run:

```
python manage.py createsuperuser
```

Enter a username, email address (if requested), and password. Password characters may not appear while typing; this is normal. Keep your password private.

Part F — Run and open the website

10. Start the development server:

```
python manage.py runserver
```

Keep the terminal open while using the website.

11. Open these addresses in your browser:

```
Website:     http://127.0.0.1:8000/
Admin page:  http://127.0.0.1:8000/admin/
```

The home page may show Django's default page. Use the **/admin/** address to manage products.

Part G — Add at least five products

12. On the admin page, sign in using the superuser username and password.

13. Open **Products** → **Add Product**. Enter the product name, category, price, and quantity, then click **Save**.

14. Repeat until at least five products have been added.

15. To edit a product, open it from the Products list, change the details, and save. To delete one, open the product and use **Delete**, then confirm.

Part H — Quick revision

• **Project:** ProductManagementSystem

• **App:** products

• **Model:** Product

• **Fields:** name, category, price, quantity

• **makemigrations:** prepares migration files from model changes

• **migrate:** applies migrations and updates the database

• **createsuperuser:** creates an administrator account

• **runserver:** starts Django's development server

• **admin.py:** registers Product so it appears in Django Admin

Common errors and fixes

“No module named django”: Install Django in the active Python environment using **python -m pip install django**.

“can't open file manage.py”: The terminal is in the wrong folder. Change directory to the folder containing manage.py.

“no such table”: Run **python manage.py makemigrations** and then **python manage.py migrate**.

Port already in use: Run **python manage.py runserver 8001** and open **http://127.0.0.1:8001/admin/**.

Admin does not show Products: Check that 'products' is in INSTALLED_APPS, Product is registered in products/admin.py, and migrations have been applied.

Final exam checklist

■ Project and app created successfully

■ Product model saved in products/models.py

■ 'products' added to INSTALLED_APPS

■ Product registered in products/admin.py

■ makemigrations and migrate completed

■ Superuser created

■ Server running and admin page opens

■ At least five products saved

■ Able to edit and delete a product